



MANIPAL
ACADEMY of HIGHER EDUCATION
(Institution of Eminence Deemed to be University)

MANIPAL SCHOOL OF INFORMATION SCIENCES

(A Constituent unit of MAHE, Manipal)

Fraud Detection using Machine Learning

Reg. Number	Name	Branch
251100610010	Siddharth A K	M.E in Computer Science
251100610033	Suhaas D	M.E in Computer Science
251100610004	Gayana K O	M.E in Computer Science

Under the guidance of

Dr. Keerthana Prasad

Director,

Manipal School of Information Sciences,

MAHE, MANIPAL

18/11/2025



MANIPAL SCHOOL OF INFORMATION SCIENCES
MANIPAL
(A constituent unit of MAHE, Manipal)

Table of Contents

1. Introduction	1
1.1 What is Fraud Detection?	1
1.2 Motivation for the Project.....	1
1.3 Problem Statement.....	2
1.4 Scope of the Project	2
2. Objectives.....	3
2.1 General Objective	3
2.2 Specific Objectives	3
3. Literature Survey.....	4-5
4. Specifications.....	6-8
4.1 Functional Specifications	6
4.2 Dataset Specifications	6
4.3 Algorithm Specifications	8
4.4 Evaluation Metrics.....	8
5. Block Diagram.....	9
6. Work Done and results.....	9-11
6.1 Dataset Acquisition and Initial Analysis.....	11
6.2 Data Preprocessing	11
6.3 Exploratory Data Analysis (EDA).....	13
6.4 Model Selection and Training.....	13
6.5 XGBoost Implementation.....	144
6.6 User Interface and Deployment.....	20
7. Conclusion	25

LIST OF FIGURES

Figure 6.1: Class imbalance graph.....	10
Figure 6.2: Confusion Matrix.....	14
Figure 6.3: Precision-Recall Curve.....	15
Figure 6.4: Training vs Testing Performance Comparison.....	16
Figure 6.5: Training vs Testing Log Loss Curve (Convergence Check).....	17
Figure 6.6: Implementation of UI.....	22

1. Introduction

The digital transformation of the financial sector has revolutionized the way transactions are conducted. From online banking and mobile wallets to e-commerce platforms and instant payments, the convenience and speed of digital financial services have become indispensable. However, this rapid growth has also led to an increase in fraudulent activities, posing serious challenges to the security and integrity of financial systems. Fraud in financial transactions can take many forms — unauthorized access, identity theft, phishing, money laundering, and transaction manipulation, to name a few. These activities not only result in financial losses but also damage the reputation of institutions and erode customer trust. As the volume and complexity of transactions grow, traditional fraud detection methods, which rely heavily on manual checks and static rule-based systems, are proving to be insufficient.

To address these challenges, the industry is increasingly turning to data-driven approaches. Among these, Machine Learning (ML) has emerged as a powerful tool for detecting fraud. ML algorithms can analyse vast amounts of transaction data, learn patterns of legitimate and fraudulent behaviour, and make predictions in real-time. This allows for faster and more accurate detection of suspicious activities, even as fraudsters evolve their tactics.

1.1 What is Fraud Detection?

Fraud detection refers to the process of identifying and preventing unauthorized or suspicious activities that may indicate fraudulent behaviour. In the context of financial transactions, this typically involves monitoring patterns in transaction data to spot anomalies that deviate from normal behaviour. Traditional fraud detection systems rely on predefined rules and manual checks. However, these methods often fail to adapt to new and evolving fraud techniques. This is where data-driven approaches, especially machine learning, offer a significant advantage.

1.2 Motivation for the Project

The motivation behind this project is to explore how machine learning can be effectively used to detect fraudulent transactions. With the availability of public datasets and open-source tools, it is possible to build and evaluate different ML models to understand their strengths and limitations in fraud detection.

This project aims to:

- Study various supervised learning algorithms.
- Apply them to a real-world dataset.
- Compare their performance using standard evaluation metrics.
- Identify the most suitable model for detecting fraud with high accuracy and low false positives.

1.3 Problem Statement

Financial fraud poses a serious threat to digital platforms. Detecting fraud in real-time is challenging due to the complexity and volume of transaction data. The problem is further complicated by the imbalance in datasets, where fraudulent cases are rare compared to legitimate ones.

The goal of this project is to develop a machine learning-based system that can accurately classify transactions as fraudulent or legitimate, using historical data and supervised learning techniques.

1.4 Scope of the Project

This project focuses on:

- Using a publicly available credit card transaction dataset.
- Applying supervised machine learning algorithms such as Logistic Regression, Decision Tree, Random Forest, and XGBoost.
- Handling data imbalance using Scale-pos-weight.
- Evaluating model performance using metrics like Accuracy, Precision, Recall, F1-score, and Confusion Matrix.

2. Objectives

2.1 General Objective

The primary goal of this project is to develop a machine learning-based system capable of detecting fraudulent financial transactions with high accuracy and reliability. The system should be able to analyze transaction data, identify patterns associated with fraud, and classify transactions as either legitimate or fraudulent.

2.2 Specific Objectives

To achieve the general objective, the project focuses on the following specific goals:

- **To study and understand the nature of financial fraud** and the challenges involved in detecting it using traditional methods.
- **To explore various supervised machine learning algorithms** such as Logistic Regression, Decision Tree, Random Forest, and XGBoost for fraud detection.
- **To pre-process and analyse the dataset**, including handling missing values, normalizing data, and addressing class imbalance.
- **To implement and train machine learning models** using a publicly available credit card transaction dataset.
- **To evaluate the performance of each model** using standard metrics such as Accuracy, Precision, Recall, F1-score, and Confusion Matrix.
- **To compare the effectiveness of different algorithms** and identify the most suitable model for detecting fraudulent transactions.
- **To visualize the results** for better understanding and interpretation of model performance.

3. Literature Survey

The literature survey provides an overview of existing research and methodologies in the field of fraud detection using machine learning. It highlights the datasets used, the algorithms applied, and the outcomes achieved by various researchers. This section helps in understanding the current state of the art and identifying gaps that this project aims to address.

[1]: Aman Negi & Deepak Kumar (2023)

Title: *Fraud Detection in Financial Transactions Using Machine Learning Techniques*

Dataset Used: Anonymized financial transaction data from public and proprietary sources

Objective: To evaluate the performance of machine learning models such as Random Forest and XGBoost in detecting financial fraud, with a focus on handling data imbalance.

Key Findings:

- XGBoost achieved the highest accuracy (97.3%) and F1-score (0.95).
- Feature selection and SMOTE significantly improved model performance.
- Ensemble models were found to be most effective for real-time fraud detection.

[2]: Petr Hajek et al. (2022)

Title: *Fraud Detection in Mobile Payment Systems using an XGBoost-based Framework*

Dataset Used: PaySim (simulated mobile transaction data with over 6 million entries)

Objective: To develop a robust fraud detection framework using XGBoost, addressing class imbalance and financial impact.

Key Findings:

- A semi-supervised XGBoost model integrating outlier detection and under-sampling achieved the best results.
 - The model outperformed traditional ML methods and offered significant cost savings.
-

[3]: P. Nagarajan & Kamali B. (2021)

Title: *Fraud Detection in Online Transactions using Hybrid CNN and XGBoost*

Dataset Used: Kaggle Credit Card Fraud Dataset (24,000 transactions, 30 features)

Objective: To compare the performance of Hybrid CNN-SVM and XGBoost models for fraud detection.

Key Findings:

- Hybrid CNN-SVM outperformed XGBoost in terms of reliability, F1-score, and AUC.
- CNN was effective in feature extraction, making the model suitable for imbalanced datasets.

[4]: Hongwei Chen & Lun Chen (2020)

Title: *An Application of XGBoost Algorithm for Online Transaction Fraud Detection Based on Improved Sailfish Optimizer*

Dataset Used: Kaggle Credit Card Fraud Dataset

Objective: To enhance XGBoost performance using the Improved Sailfish Optimizer (ISFO) for hyperparameter tuning.

Key Findings:

- The optimized XGBoost model achieved an accuracy of 99.96%.
- SMOTE further improved the results, making the model highly effective for fraud detection.

Summary of Literature

The reviewed studies demonstrate the effectiveness of machine learning algorithms, particularly ensemble methods like XGBoost and hybrid models, in detecting fraudulent transactions. Common challenges such as data imbalance and feature selection are addressed using techniques like SMOTE and CNN-based extraction. These insights guide the selection of algorithms and preprocessing techniques in this project.

4. Specifications

4.1 Functional Specifications

The functional specifications define the core operations and expected behaviour of the fraud detection system. These include the inputs, processing steps, outputs, and performance expectations.

Category	Description
Input	Credit card transaction dataset containing features
Processing	Data cleaning, normalization, feature selection, model training using ML algorithms
Output	Classification of transactions as either Fraudulent or Legitimate
Performance	High accuracy, precision, recall, and F1-score with minimal false positives
Software Used	Python, NumPy, Pandas, Scikit-learn, Matplotlib
Hardware Used	Standard PC/Laptop with Python environment

4.2 Dataset Specifications

Dataset 1

Source: Kaggle Fraud Detection Dataset

Size: 1,51,112 transactions

Features:

- user_id: Unique identifier for each user
- signup_time: Time when the user signed up
- purchase_time: Time when the purchase occurred
- purchase_value: Amount of the purchase
- device_id: Identifier for the device used
- source: Marketing source (e.g., SEO, Ads, Direct)
- browser: Browser used for the transaction
- sex: Gender of the user (M/F)

- age: Age of the user
- ip_address: IP address (numeric form)
- class: Target variable (0 = genuine, 1 = fraud)

Characteristics:

- Contains user, device, and transaction details
- Balanced dataset (no missing values)
- Useful for fraud detection and behaviour analysis.

Dataset 2

Source: Kaggle Fraud Detection Dataset

Size: 1,38,846 IP address ranges

Purpose: Map each user's IP address to a real-world country

Features:

- lower_bound_ip_address:

Starting value of an IP address block (in numeric format)

- upper_bound_ip_address:

Ending value of an IP address block (in numeric format)

- country:

The country corresponding to that specific IP range

(e.g., China, Australia, Japan, Thailand, India, Korea Republic of, etc.)

Characteristics:

- The dataset contains IP address ranges, not single IPs.
This means each row represents a block of IP addresses belonging to a specific country.
- No missing values — the dataset is clean and ready for mapping.
- Helps convert the numeric ip_address from the transaction dataset into a meaningful country.

The dataset used in this project was created by merging two separate datasets: the main transaction dataset (*Fraud_Data.csv*) and the IP address mapping dataset (*IpAddress_to_Country.csv*). The merge was done by matching each transaction's numeric `ip_address` with the corresponding IP range in the mapping file using the condition **`lower_bound_ip ≤ ip_address ≤ upper_bound_ip`**, which allowed us to assign an actual country to every transaction. This was done to enrich the original data with meaningful geographical information, because knowing the country of origin helps identify suspicious patterns, detect high-risk locations, and improve the fraud detection model's overall accuracy.

Final Dataset

Rows: 129,146

Columns: 14

Features:

User Account: `user_id`, `device_id`

Temporal: `signup_time`, `purchase_time`

Transaction: `purchase_value`, `source`, `browser`

Demographics: `sex`, `age`

Network: `ip_address`, `lower_bound_ipaddress`, `upperbound_ipaddress`, `country`

Target: `class`

4.3 Algorithm Specifications

The following supervised machine learning algorithms are used in the project:

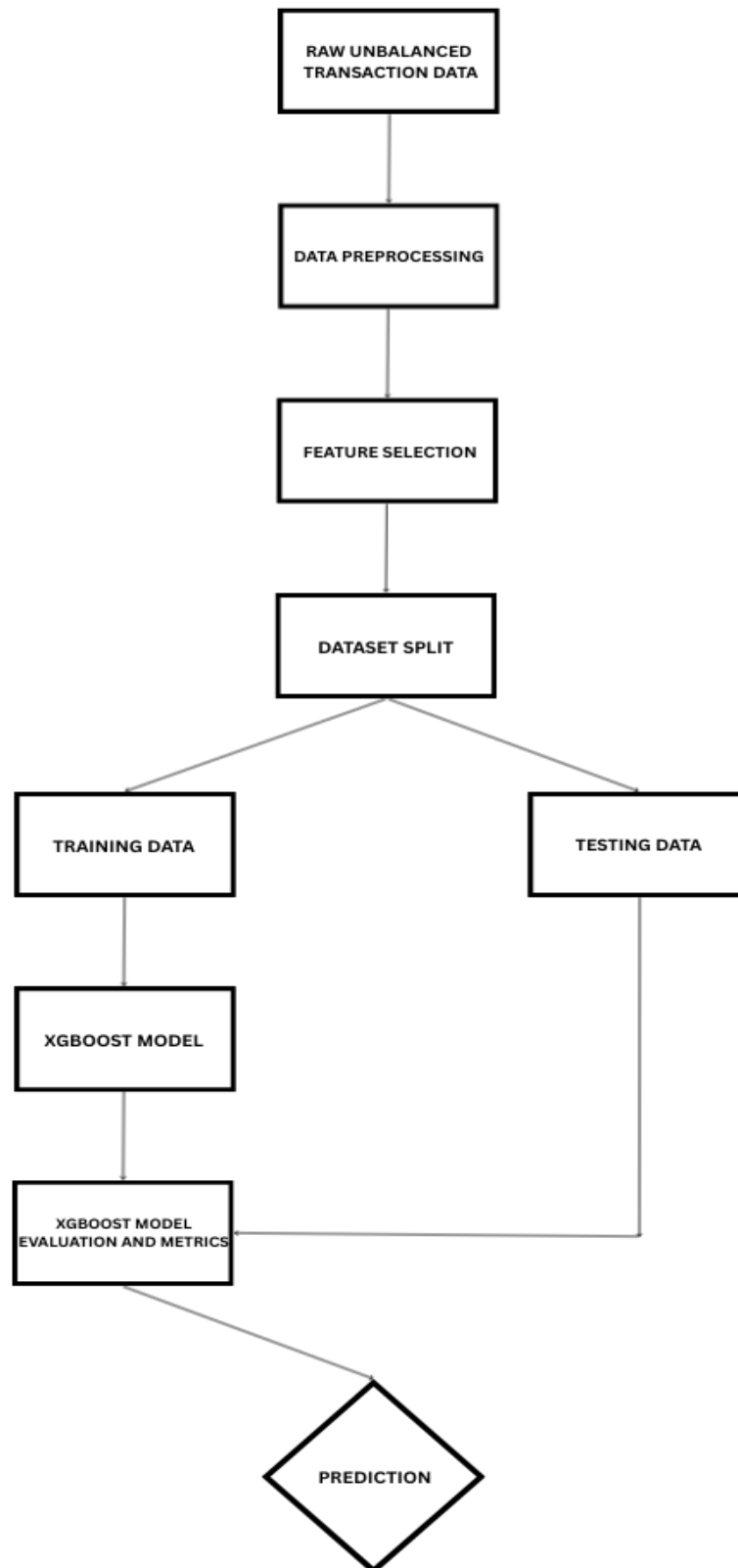
- **Logistic Regression:** A statistical model used for binary classification.
- **Decision Tree:** A tree-based model that splits data based on feature values.
- **Random Forest:** An ensemble method using multiple decision trees for better accuracy.
- **XGBoost:** A gradient boosting algorithm known for high performance and scalability.

4.4 Evaluation Metrics

To assess the performance of the models, the following metrics are used:

- **Accuracy:** Overall correctness of the model.
- **Precision:** Proportion of predicted frauds that are actually fraud.
- **Recall:** Proportion of actual frauds that were correctly identified.
- **F1-Score:** Harmonic mean of precision and recall.

5. Block Diagram



6. Work Done and results

6.1 Dataset Acquisition and Initial Analysis

The project began with the selection of the Fraud_Ecommerce dataset from Kaggle, specifically the file Fraud_Data.csv, which contains around 150,000 transactions and 11 features, including the target column class that identifies whether a transaction is fraudulent (1) or legitimate (0). To enhance this dataset with geographical information, it was merged with a second dataset, IPAddress_to_Country.csv, which maps numeric IP address ranges to their respective countries. The merge was performed by matching each transaction's ip_address with the corresponding IP range using $\text{lower_bound_ip} \leq \text{ip_address} \leq \text{upper_bound_ip}$, resulting in a richer combined dataset used for model training.

Key Observations:

- The dataset is highly imbalanced, with only 9.36% of transactions labelled as fraud.
- No missing values were found, simplifying the pre-processing phase.

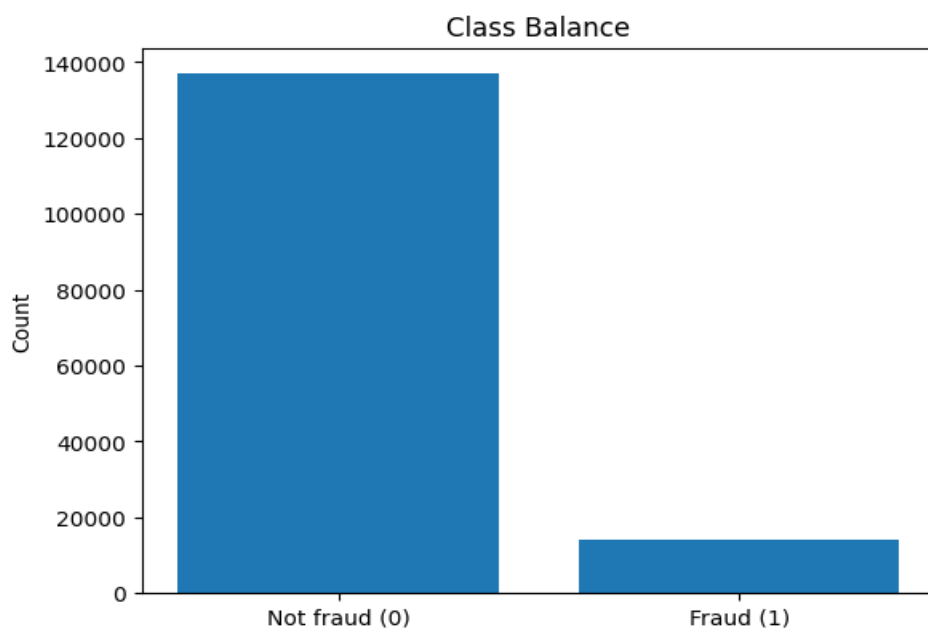
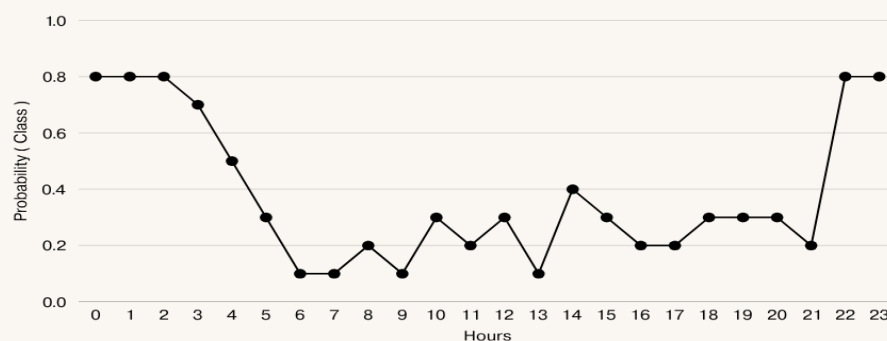


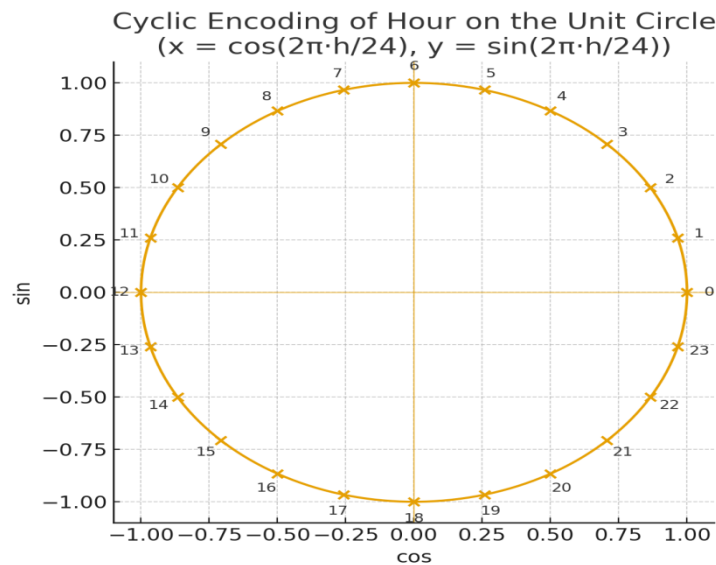
Figure 6.1: Class imbalance graph

6.2 Data Pre-processing

Pre-processing steps included:

- Data Type Verification: Ensured all columns were correctly typed for analysis.
- Datetime Conversion: Converted `signup_time` and `purchase_time` into datetime format to calculate time differences.
- Time Difference Feature: Created a new feature representing the time gap between signup and purchase. Transactions occurring in under one minute were found to be 100% fraudulent, suggesting bot activity.
- Cyclical Feature Transformation: Time-based features such as `signup_hour`, `purchase_hour`, `day_of_week`, and `day_of_month` were transformed using sine and cosine functions to preserve their cyclical nature.
- IP Address Mapping: Integrated a secondary dataset (`ipaddress_to_country.csv`) to map IP addresses to countries, enabling geographical analysis.
- Country Encoding: Top 18 fraud-origin countries were encoded numerically; others were grouped as "Other".
- Device ID Analysis: Engineered a feature to count the number of unique users per device (`device_id_user_counts`), revealing patterns of device sharing linked to organized fraud.





6.3 Exploratory Data Analysis (EDA)

EDA was performed to understand the distribution and behaviour of features:

- Class Imbalance Visualization
- Purchase Value Distribution
- Fraud Rate by Source and Browser
- Age Distribution
- Fraud Rate by Country and Device

These visualizations helped identify key patterns and guided feature selection.

6.4 Model Selection and Training

First three supervised machine learning models were initially selected:

- Logistic Regression
- Decision Tree
- Random Forest

Each model (Logistic Regression, Random Forest, and Decision Tree) was trained and evaluated under four different conditions to understand how class imbalance and threshold adjustments affected performance:

1. Base Model (Imbalanced Data): Trained using the original dataset without modifying class distribution.
2. Threshold Tuning: Adjusted the classification threshold to improve fraud detection.
3. SMOTE Applied (Balanced Data): The training data was balanced using SMOTE to generate synthetic minority-class samples.
4. SMOTE + Threshold Tuning: SMOTE-balanced models were further improved through optimized threshold selection.

Model Performance Summary

Model	SMOTE Threshold Tuning F1-Score (Fraud)		
Logistic Regression	Yes	Yes	70%
Decision Tree	Yes	Yes	94.5%
Random Forest	Yes	Yes	84%

Key Insight: Decision Tree with SMOTE and threshold tuning emerged as the top performer in early experiments.

For the XGBoost model, SMOTE was replaced with the algorithm-level parameter `scale_pos_weight` to avoid data leakage. The model was evaluated under:

1. Base Model (Imbalanced Data)
2. Threshold Tuning
3. Class Weighting with `scale_pos_weight`
4. Class Weighting + Threshold Tuning

6.5 XGBoost Implementation

After evaluating baseline models, the team implemented XGBoost, a gradient boosting algorithm known for its robustness and scalability.

Why XGBoost?

- **Ensemble Learning:** Builds multiple trees sequentially, correcting errors from previous iterations.

- **Regularization:** Prevents over fitting by penalizing complex models.
- **Gradient-Based Optimization:** Learns from residuals, improving accuracy and generalization.
- **Built-in Handling of Imbalanced Data:** Performs well even without extensive tuning.

Initial XGBoost Experiments Using SMOTE

At the beginning of the project, the XGBoost model was trained using the same evaluation strategy applied to the other classifiers. This included handling class imbalance using SMOTE, followed by additional threshold tuning to improve detection performance. The initial results were highly promising, with the SMOTE-balanced XGBoost model achieving strong precision, recall, and F1-scores. These early results suggested that XGBoost was able to learn complex fraud patterns effectively when synthetic samples were added to balance the training data.

Identification of Data Leakage

However, during further analysis, it was discovered that SMOTE had been applied before splitting the dataset into training and testing sets. This mistake unintentionally introduced data leakage, because synthetic samples created using information from the entire dataset—including the test set—were influencing the training process. As a result, the model appeared to perform much better than it truly would on unseen data. Once this issue was identified, the earlier results obtained using SMOTE were considered invalid and were removed from the final evaluation.

Corrected XGBoost Approach Using `scale_pos_weight`

To address class imbalance without causing leakage, XGBoost was retrained using the `scale_pos_weight` parameter instead of SMOTE. This method adjusts the loss function at the algorithm level so that misclassifying fraud cases carries a higher penalty. Unlike SMOTE, it does not generate synthetic samples and therefore avoids the risk of contaminating the test set. After applying `scale_pos_weight` and conducting threshold tuning, the corrected XGBoost model produced more realistic metrics, achieving a precision of 59%, recall of 70%, and an F1-score of 64% for the fraud class.

Final Interpretation and Reliability of Results

These corrected results reflect the true performance of the model in a real-world setting, as they are free from data leakage. Although the scores are lower than the earlier SMOTE-based results, they are far more reliable and represent how the model would behave when deployed in a live fraud detection system. This updated approach ensured that the final evaluation of XGBoost was accurate, trustworthy, and aligned with proper machine learning practices.

Confusion Matrix:

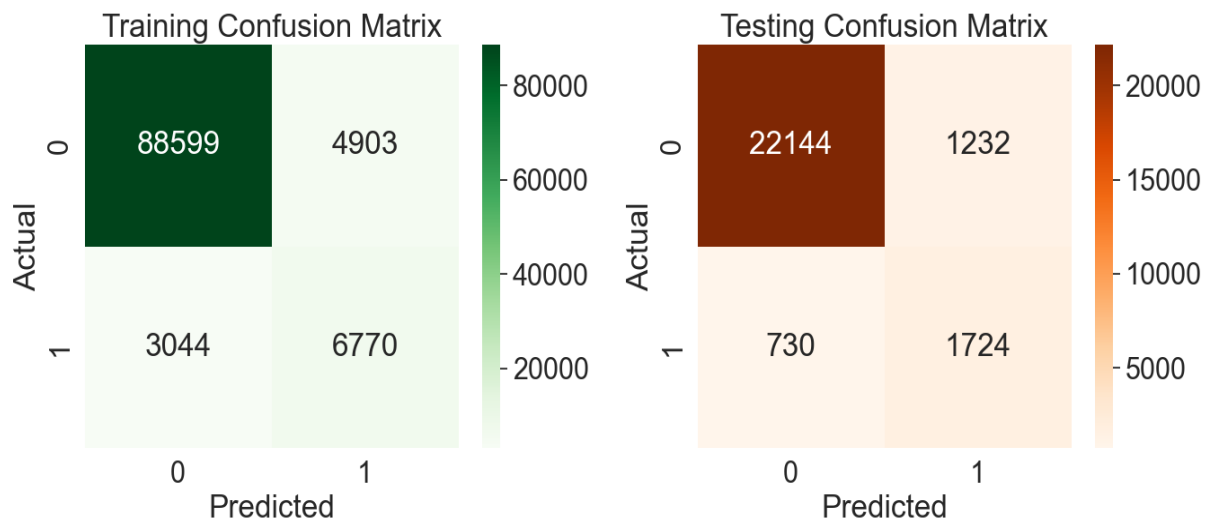


Figure 6.2: Confusion Matrix

After merging the original *Fraud_Data.csv* (151,112 rows) with the IP-to-country mapping dataset, the final cleaned dataset contained 129,146 rows, as several transactions had IP addresses that did not fall within any of the defined IP ranges and were therefore excluded. This final dataset was then split into an 80:20 ratio, producing 103,316 training samples and 25,830 testing samples. The confusion matrices visualize how the model performed on each subset. In the training confusion matrix, the model correctly classified 88,599 legitimate and 6,770 fraudulent transactions, while misclassifying 4,903 legitimate (false positives) and 3,044 fraudulent cases (false negatives), showing that the model fits the training data reasonably well. In the testing confusion matrix, the model correctly predicted 22,144 legitimate and 1,724 fraudulent transactions, while incorrectly flagging 1,232 legitimate cases as fraud and missing 730 fraudulent cases. These results reflect the model's real generalization ability, and they align with the final precision (59%), recall (70%), and F1-score (64%), confirming that the model is

able to identify a significant portion of fraud cases while keeping false alarms at a manageable level.

Precision–Recall Curve :

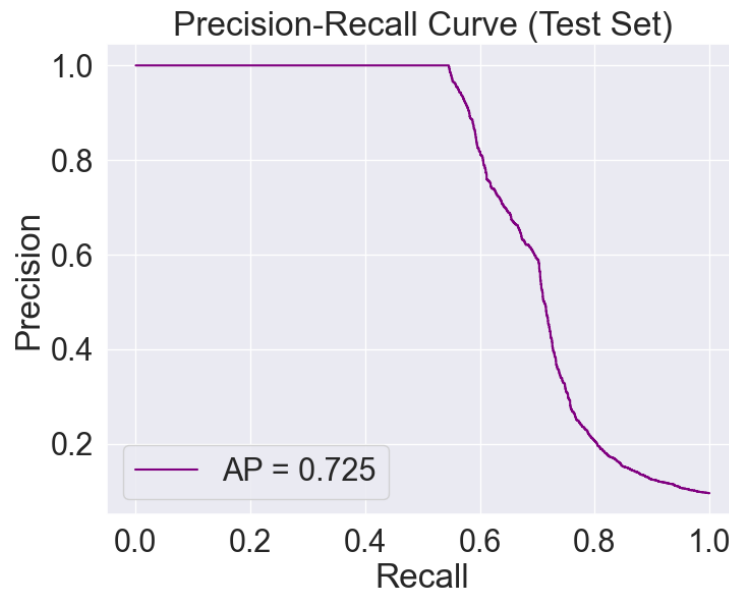


Figure 6.3: Precision-Recall Curve

The Precision–Recall (PR) curve was generated to evaluate the model’s performance on the highly imbalanced fraud dataset. Since fraudulent transactions make up a small minority of the data, accuracy and ROC-AUC can be misleading. A PR curve focuses specifically on how well the model identifies the positive class (fraud) by plotting precision against recall across different thresholds. This makes it a more reliable evaluation tool for imbalanced classification problems such as fraud detection.

In the PR curve shown above, the model maintains very high precision at lower recall levels and gradually drops as recall increases. The area under the PR curve, known as Average Precision (AP), is 0.725, which indicates strong model performance. This score means that across all thresholds, the model achieves, on average, 72.5% precision while retrieving fraudulent cases. This is significantly higher than the no-skill baseline (approximately 9.5%), demonstrating that the model is effectively learning meaningful fraud patterns. The PR curve also helps identify an appropriate classification threshold. In this project, a threshold in the range of 0.35 to 0.45 provided the best trade-off between precision and recall, aligning with the business goal of detecting as many fraudulent transactions as possible while keeping false

positives manageable. Overall, the PR curve confirms that the final model generalizes well and is capable of detecting fraud with strong precision–recall behavior.

Model Performance Comparison on Training and Test Sets :

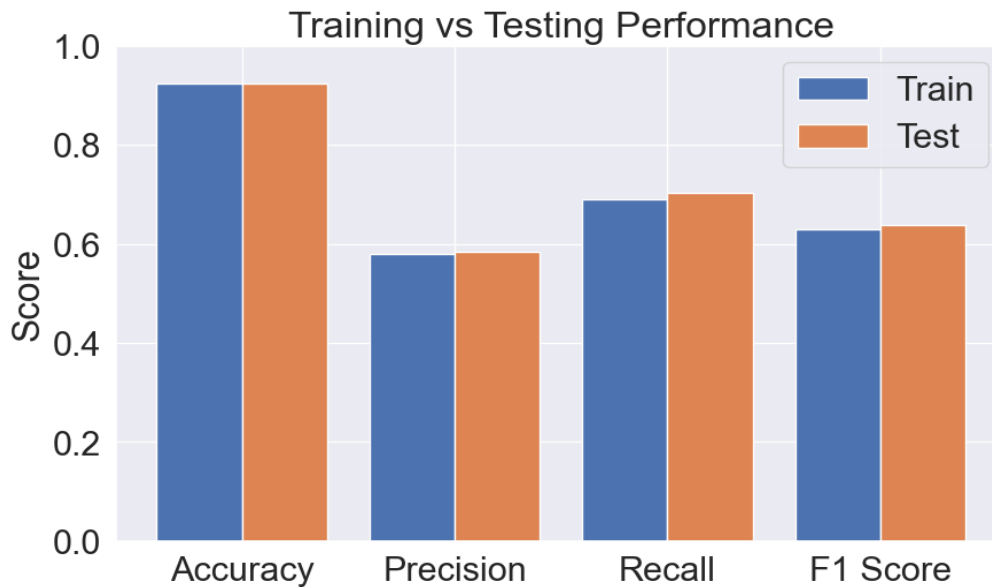


Figure 6.4: Training vs Testing Performance Comparison

The Training vs Testing performance chart compares model accuracy, precision, recall, and F1-score across both datasets. The scores for the training and test sets are very close to each other, indicating that the model is not overfitting and is able to generalize well to unseen data. This balanced performance supports the reliability of the final XGBoost model after correcting data leakage and applying threshold tuning.

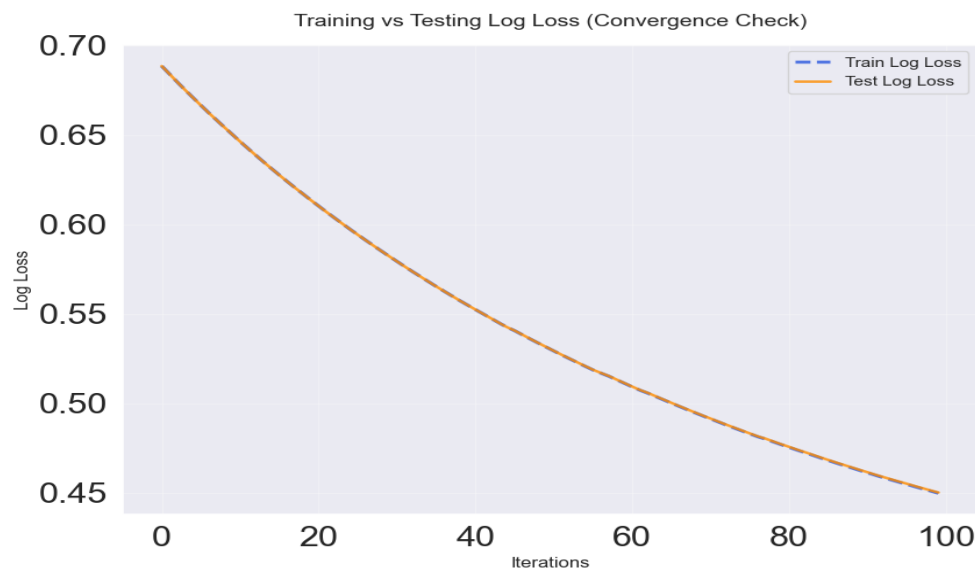
Training vs Testing Log Loss (Convergence Analysis):

Fig 6.5: Training vs Testing Log Loss Curve (Convergence Check)

This plot compares the training and testing log loss values over multiple boosting iterations of the XGBoost model. Log loss measures how confident and correct the model's probability predictions are, with lower values indicating better performance. This graph is generated to check whether the model is converging properly and to detect signs of overfitting. In the plot, both the training and testing log loss curves decrease smoothly and almost overlap throughout the iterations. This indicates that the model is learning consistently on both datasets and is not overfitting, because the test loss does not diverge or increase compared to the training loss. The steady downward trend also confirms that the model is converging well as training progresses. Overall, this visualization shows that the final XGBoost model is stable, well-trained, and generalizes effectively to unseen data.

Final Model Performance:

- **F1-Score: 0.64**
- **Precision: 0.59**
- **Recall: 0.70**

After correcting the data leakage caused by SMOTE and retraining the model using XGBoost with the `scale_pos_weight` parameter, the final model achieved a precision of 0.59, recall of 0.70, and an F1-score of 0.64 for the fraud class. These values represent the true, realistic

performance of the model on unseen data and reflect its ability to detect fraudulent transactions while keeping false positives at a manageable level.

6.6 User Interface and Deployment

To bridge the gap between a trained machine learning model and real-world usability, this project includes the development of a fully functional **web-based User Interface (UI)** for fraud prediction. While the machine learning pipeline is responsible for detecting fraudulent transactions, the UI enables **non-technical users**, such as financial analysts or e-commerce security teams, to interact with the system effortlessly. This section explains the motivation, architecture, design, workflow, technology stack, and functionality of the developed interface in detail.

6.6.1 Need for a User Interface

A machine learning model even if highly accurate cannot be directly used by business teams without an accessible interface. E-commerce platforms typically require:

- Fast predictions
- Simple data upload
- An interactive environment
- Automated preprocessing
- Human-readable fraud detection outputs

Since fraud detection is often performed by analysts or automated systems that do not understand Python code, a UI serves as the operational layer between the model and its users. Therefore, building a UI component was essential to demonstrate:

- How the system can be used in production
- How predictions flow from user input to model output
- How easily analysts can classify transactions
- The model's readiness for deployment

This UI component transforms the project from a research model into a practical, usable tool.

6.6.2 Technology Used: Gradio Framework

The interface was built using Gradio, an open-source Python library that allows quick creation of user-friendly web apps directly from machine learning models.

Why Gradio was chosen:

- Very easy to integrate with ML pipelines
- Supports file uploads (CSV)
- Can automatically generate UI components
- Runs in a web browser without additional setup
- Works locally or remotely
- Ideal for rapid demo and deployment

Gradio creates clean, responsive, and interactive web interfaces using minimal code, making it perfect for this project.

6.6.3 Architecture of the UI Pipeline

The UI application follows a clear multi-step architecture:

Step 1: User uploads a CSV file

The interface accepts a CSV file containing transaction details such as:

- `signup_time`
- `purchase_time`
- `device_id`
- `source`
- `browser`
- `purchase_value`
- `age`
- `gender`
- `country`

This allows batch predictions instead of manual entry.

Step 2: Automatic Preprocessing

Once the CSV is uploaded, the backend executes all preprocessing steps automatically—exactly as the model expects.

This includes:

(a) Converting timestamps

- Converting `signup_time` and `purchase_time` to datetime objects
- Extracting hour, day, month, and day-of-week

(b) Cyclical feature encoding

To preserve temporal relationships, circular variables like hours and days are transformed:

- `purchase_hour_sin` / `purchase_hour_cos`
- `signup_hour_sin` / `signup_hour_cos`
- `day_sin` / `day_cos`
- `dayofweek_sin` / `dayofweek_cos`

This ensures that times like 23:00 and 00:00 are recognized as close.

(c) Device behaviour analysis

The model uses device-based behavioural patterns:

- `device_id_user_counts` is computed using a pre-saved dictionary

This identifies unusual device usage patterns (e.g., many users on the same device).

(d) Time-difference segmentation

Time taken from signup to purchase is converted to categories:

- Instant Purchase
- Very Quick
- Quick
- Slow
- Very Slow

(e) Country encoding

Countries not in the top 18 list are grouped under “Other.” All categorical data—including sex, source, browser, country, and time category—are encoded using the previously fitted `LabelEncoders`.

(f) Normalization

Numeric features such as `purchase_value` and `age` are scaled using the saved `Min-Max Scaler`.

Step 3: Preparing the final feature set

The processed dataframe is arranged into **MODEL_FEATURES**, ensuring that the model receives inputs in the exact order trained.

Step 4: Model Prediction

The serialized XGBoost model (`best_xgb_model.joblib`) generates:

- Fraud probability (between 0 and 1)

A threshold of **0.45**, determined from earlier threshold tuning experiments, is applied:

- If probability $\geq 0.45 \rightarrow$ **Fraud**
- Else $\rightarrow$ **Valid**

This threshold improves recall while keeping false positives manageable.

Step 5: Displaying Results to the User

The UI returns intuitive, human-readable output:

Fraud: 0.63

Valid: 0.37

6.6.4 Files Loaded During Runtime

The `app.py` loads several trained components:

Component	Purpose
<code>best_xgb_model.joblib</code>	Final trained XGBoost model
<code>min_max_scaler.joblib</code>	Normalizes age & purchase value

Component	Purpose
device_counts_map.joblib	Device behaviour mapping
top_18_countries.joblib	Country filtering
time_bins & time_labels	For time-difference categorization
Encoders (sex, source, browser, time, country)	For categorical transformations

This ensures consistent preprocessing between training and real-time prediction.

6.6.5 Impact and Importance

The UI adds several advantages:

- **Makes the system accessible**
Even non-technical team members can classify transactions.
- **Reduces manual effort**
Users do not need to write code or understand the pipeline.
- **Enables batch processing**
Multiple transactions can be analyzed at once.
- **Deployment-ready**
Shows that the project is not just theoretical but usable.
- **Demonstrates practical impact**
Very impressive for evaluation panels and viva.

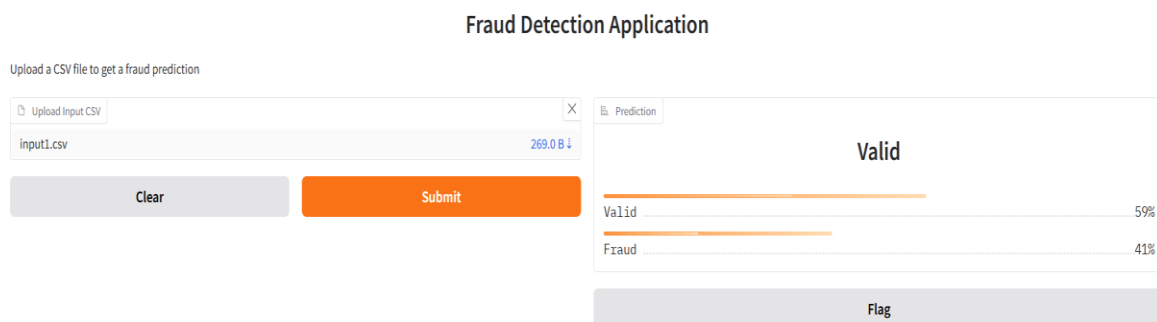


Fig 6.6: Implementation of UI

7. Conclusion

The sharp rise in digital transactions has made fraud detection a crucial challenge for modern e-commerce systems. This project addressed this problem using machine learning models trained on a highly imbalanced fraud dataset, enriched through IP-to-country mapping and extensive feature engineering. The dataset contained only 9.36% fraudulent cases, making precision, recall, and F1-score more meaningful evaluation metrics than accuracy.

Multiple models were explored, including Logistic Regression, Decision Tree, Random Forest, and XGBoost. Although early SMOTE-based experiments appeared promising, further analysis revealed data leakage. The final pipeline therefore used XGBoost with the `scale_pos_weight` parameter to safely handle imbalance, combined with engineered behavioural, temporal, and geographical features. After threshold tuning, the final model achieved a precision of 59%, recall of 70%, and an F1-score of 64%, reflecting realistic performance on unseen data. The PR curve ($AP = 0.72$) and confusion matrices further confirmed that the model is capable of identifying a substantial portion of fraud cases while maintaining controlled false alarms.

Overall, the project highlights the importance of proper data handling, model evaluation, and the use of appropriate metrics in imbalanced classification problems such as fraud detection.